MDPI

Article

Seeing the Process Through the Worker's Eyes: Event Logs from Egocentric Factory Videos

Alessandro Berti [1,*] and Wil M. P. van der Aalst [1,2]

[1] Process and Data Science (PADS), RWTH Aachen University, Aachen, Germany; a.berti@pads.rwth-aachen.de (A.B.); wvdaalst@pads.rwth-aachen.de (W.M.P.v.d.A.)
[2] Celonis, Munich, Germany
* Correspondence: a.berti@pads.rwth-aachen.de

Abstract

Much of manual factory work remains invisible to information systems: moving materials, preparing tools, checking surfaces, and repairing defects can leave no event record. Egocentric video brings this work into view, but turning what a camera sees into a process log requires decisions about actions, names, and time. We present a construction method and linked data resource that make these decisions inspectable. A vision-language model produces timestamped descriptions of visible work, which we call generated transcripts. Pretrained models summarize these descriptions, derive shared factory labels, and extract labeled event intervals. The resulting logs support exploration of work composition, local sequences, and differences between recorded worker profiles. Structural checks and an author audit assess the records and their consistency with generated transcripts, while accuracy against the source videos remains an open question. Preserving the intermediate descriptions alongside the logs lets researchers trace measurements to their textual basis and investigate alternative representations of manual work.

Keywords: process mining; inferred event logs; egocentric video; measurement; data quality; manufacturing; foundation models

1. Introduction

Process mining uses timestamped event records to study operational processes [1]. Many manual factory tasks leave no such records. An information system may record an order or machine checkpoint while missing how a worker loads parts, moves material, checks a component, or switches tools. Video can make some of this work observable, but it does not directly provide the case identifiers, labels, and event boundaries that process mining needs [2].

Egocentric video records work from the camera wearer's perspective. The public Egocentric-10K resource provides factory recordings of this kind [3]. Figure 1 illustrates the viewpoint. Turning recordings into event logs requires decisions about what constitutes an event, which names to use, and how to interpret time. These are *measurement decisions* as well as extraction tasks: they determine which aspects of work a log can represent and which questions an analysis can answer.

Prior work has shown how video can support workflow understanding [4,5] and event-log construction [6]. Building on this work, we ask how recordings of manual factory work can be transformed into a *shared event representation* whose construction and analytical meaning remain inspectable. Shared activity names allow observations from different

Received:
Revised:
Accepted:
Published:

Copyright: © 2026 by the authors. Submitted to *Systems* for possible open access publication under the terms and conditions of the Creative Commons Attri- bution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).



Figure 1. Example frames from Build AI's Egocentric-10K resource [3], illustrating the first-person view of manual factory work.

recordings to be considered together. Understanding the resulting measurements also requires a clear account of how observations were selected, described, and grouped.

We present a construction method and a linked data resource for retrospective analysis of egocentric factory video. Pretrained models produce timestamped descriptions of visible work, which we call *generated transcripts*, and use them to derive shared labels and event logs. Preserving the intermediate descriptions makes it possible to trace the resulting events to their textual basis. Figure 2 gives an overview of the approach.

The paper makes three contributions:

- **A construction method.** We describe how pretrained models transform video descriptions into event logs with shared labels, making the representation choices explicit.
- **A reusable resource.** We release linked descriptions, event logs, and analysis artifacts that support inspection and reuse of the construction process.

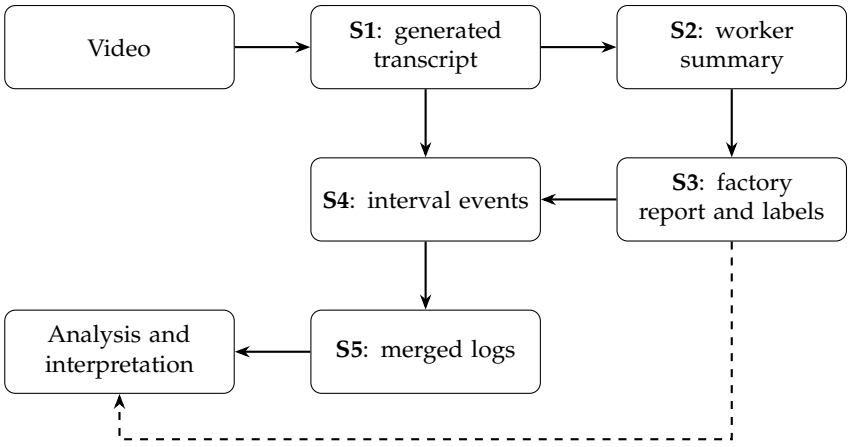


Figure 2. Overview of the construction method, from factory videos to event logs and process analysis. The dashed arrow indicates the use of shared factory labels in analysis.

Table 1. Research questions and scope.

Research question	Focus
RQ1: How are the inferred logs constructed, and which structural properties do they satisfy?	Construction and structural quality
RQ2: How well do event timing and meaning agree with the generated descriptions?	Consistency of the inferred records
RQ3: Which descriptive analyses do the logs support, and what limits their operational interpretation?	Analytical use and scope
RQ4: How do representation choices affect the interpretation of process measurements?	Measurement and interpretation

- **An assessment of the resulting representation.** We examine the records' consistency, their use in descriptive process analysis, and the limits of interpreting inferred events as measurements of work.

Four research questions organize the study (Table 1). They connect event construction to assessment, analytical use, and interpretation. The assessment distinguishes consistency with generated descriptions from accuracy against the source videos; the latter remains an open question.

The rest of the paper is organized as follows. Section 2 reviews the background and related work. Section 3 defines the problem and event representation. Section 4 presents the construction method, and Section 5 describes the dataset and released artifacts. Section 6 explains the assessment methodology, followed by the results in Section 7. Section 8 discusses the implications for process measurement. Section 9 concludes the paper with the main findings, limitations, and directions for future work.

2. Background and Related Work

2.1. Unstructured observations as process data

König et al. [7] review how unstructured data enters process mining, including the construction of logs from text, images, and video. Diba et al. [8] distinguish extraction, correlation, and abstraction. These tasks require decisions about which observations become events, how events are grouped, and which concepts describe them. Andrews et al. [9] make data quality part of construction in RDB2Log, which helps analysts derive logs from relational data. Together, these perspectives frame log construction as the design of an event representation whose quality must be assessed in relation to its intended use.

2.2. Video and first-person workflow understanding

Petersen et al. [4] model and track manual workflows from first-person video to support augmented-reality instructions and feedback during task execution. Pham and Xiao [5] extract assembly steps and operations with semantic descriptions, without pre-labeled training data or pre-trained classifiers. These studies establish first-person workflow understanding as an antecedent. Their goals include following a demonstrated task and recovering its steps.

Retrospective process analysis asks how observations across recordings can support a common description of work. This requires event representations that can be interpreted across tasks and participants. The distinction concerns the analytical objective as well as the camera viewpoint: following a task and analyzing a collection of recorded activities place different demands on the representation.

Video process mining also has several established designs. Knoch et al. [13] use hand trajectories and clustering at a camera-equipped assembly workstation. Kratsch et al. [2] develop a reference architecture, and Lepsien et al. [14] combine object tracking, action detection, and abstraction. SnapLog [11] uses pretrained visual representations, temporal segmentation, and a classifier adapted with labeled examples. Its evaluation includes TUM Kitchen and egocentric EPIC-KITCHENS recordings. Thus, camera placement and super-

Table 2. Closest construction methods: purpose, input, vocabulary, adaptation, and output. "Not reported" identifies information not established by the inspected source, not an absent capability.

Method	Purpose and input	Vocabulary origin	Task-specific adaptation	Output
Vid2log [6]	Construction and enrichment from video, with conventional logs	Domain and activity prompts	Pretrained LLaVA; domain prompts; parameter updates not reported	Combined event log
Kecht et al. [10]	Construction from customer-service conversations	Supplied topics and activities	Pretrained NLI; labeled threshold calibration and hypothesis selection	XES log
SnapLog [11]	Event discovery from video	Classes supported by labeled examples	Pretrained visual models; classifier adaptation	Events with labels or label distributions
Seeth et al. [12]	Construction from textual reports	Allowed activities from source logs	Pretrained LLMs; supervised fine-tuning compared with prompting	XES traces
This work	Construction from egocentric factory video	Shared labels derived from recordings	Fixed pretrained models; context and vocabulary induction	Event logs and linked descriptions

vision must be compared for individual methods. The sensor-data review by Brzychczy et al. [15] places these efforts within the broader problem of converting observations into process events.

2.2.1. From video descriptions to process events

Dense video captioning already combines temporal localization with natural-language descriptions [16]. Timestamped narration alone is therefore an established output. Process analysis requires further choices: reusable activity names across recordings, event granularity, selection of operationally relevant episodes, observational case identifiers, source links, and rules for interpreting durations and frequencies.

The unit of a natural-language description need not coincide with the unit required for process analysis. A description can cover several actions, while descriptions of similar actions can use different words. Event construction therefore requires both temporal and semantic abstraction. These choices determine what event counts, durations, and sequences mean.

2.3. Video-to-log and language-model-based construction

Gavric et al.'s Vid2log [6] uses LLaVA to infer a domain and relevant activities from selected frames, obtain textual event information, and combine video-derived data with conventional logs through temporal and semantic matching. It studies log richness and an assembly use case. Both Vid2log and the present work use natural language to connect visual observations to event data. Our focus is on deriving a shared vocabulary from a collection of recordings and preserving the descriptions used to construct the events.

Kecht et al. [10] use pretrained natural language inference to assign supplied topics and activities to customer-service messages. They use labeled messages to select classification thresholds and refine hypotheses, without training a new language model. Message identifiers and timestamps are retained. This illustrates an important distinction among construction methods: event names and times may be supplied by the source, selected by an analyst, or inferred from the observations.

Shirali et al. [17] apply LLMs to physical observations: they abstract IoT readings into high-level events and integrate logs from multiple sensor sources. LLM use for physical-process data therefore predates this study. Work on process-mining assistants [18] and database-query generation [19] addresses neighboring tasks with different inputs and outputs.

Table 3. Sources of temporal information, case relationships, and evaluation references in the compared methods.

Method	Origin of temporal information	Origin of case/entity relationships	Evaluation reference
Vid2log [6]	Frame positions and prompted temporal relations	Temporal/semantic matching and resource-based case inference	Assembly case study and log-richness assessment
Kecht et al. [10]	Recorded message timestamps	Supplied message and conversation identifiers	Labeled messages for classification; process discovery demonstration
SnapLog [11]	Segmented frame sequences	Video sequences as traces	Video datasets with action and boundary annotations
Seeth et al. [12]	Timestamps extracted from generated reports	Report-to-trace pairing from source logs	Source traces and process-level comparisons
This work	Inferred intervals relative to recordings	Recording and worker identifiers	Consistency with generated descriptions and structural checks

Seeth et al. [12], an arXiv preprint, pair generated reports with source traces, supply permitted activities and attributes, and evaluate extracted traces and process models against source-log references. They report benefits from fine-tuning over zero-shot and few-shot prompting in that setting. This provides a comparison in which the vocabulary and evaluation reference come from existing logs. The present work uses fixed pretrained models and derives shared labels from recordings. Differences in supervision and reference data must be considered when comparing the resulting assessments.

Tables 2 and 3 summarize these differences. They distinguish how a method obtains its vocabulary and adapts its models from how it establishes temporal information, event relationships, and evaluation references.

2.4. Event abstraction and semantic consistency

Event abstraction means choosing the level of detail represented by an event. The review by van Zelst et al. [20] explains how this choice changes process granularity and compares methods by supervision, input, and uncertainty. Three questions are relevant to construction from video: where an episode begins and ends, which reusable activity classes describe the work, and which class is assigned to each episode. Separating these questions helps distinguish temporal segmentation, vocabulary design, and event classification.

Vocabulary design and event classification can fail in different ways. A relevant concept may be absent from the vocabulary, or an available concept may be assigned to the wrong observation. Further aggregation introduces another interpretive choice by grouping detailed labels into broader categories. Semantic consistency therefore concerns both the meaning assigned to individual events and the meaning retained when those events are analyzed together.

2.5. Construction, enrichment, and integration

Text2EL+ [21] enriches existing logs with events and attributes extracted from text, using semantic and contextual validation and expert guidance. Sturm et al. [22] instantiate parsing, abstraction, extraction, integration, and enrichment using CRM data, email metadata, and call transcripts. Their evaluation involves a sales process and a domain expert. Table 4 separates the tasks these designs can combine.

The present work primarily concerns construction: it derives an initial event representation from recordings. This differs from adding information to an existing log or resolving correspondences among independently recorded sources. Intermediate descriptions can make a construction process easier to inspect, but agreement between representations derived from the same observations does not provide independent corroboration.

Table 4. Neighboring event-data tasks, distinguished by their starting points and intended results. A system can perform more than one task.

Task	Starting point	Intended result
Construction [8]	Observations or documents without a suitable log	An initial event representation
Enrichment [21]	An existing log and additional information	Added or corrected events and attributes
Integration [17,22]	Multiple sources describing related behavior	A combined representation with resolved correspondences

2.6. Quality, uncertainty, and relational representations

Suriadi et al. [23] organize recurring event-log imperfections. For logs constructed from unstructured observations, quality depends on both the source evidence and the transformations applied to it. Assessment must therefore consider how information can be omitted, altered, or propagated before it appears in an event record.

Quality annotations and explicitly uncertain events serve different purposes. Goel et al. [24] retain data-quality information for use during analysis. Pegoraro et al. [25] represent uncertainty explicitly and compute conformance bounds. SnapLog can retain candidate-label distributions [11]. These approaches distinguish documenting known quality issues from representing alternative event interpretations. Consistency checks alone do not establish calibrated probabilities or bounds on downstream error.

Esser and Fahland [26] represent events and entities across multiple dimensions in graph databases. This complements the case-choice concerns of object-centric process mining [27]. Such representations make the relationships among events explicit, but those relationships still require supporting observations. The range of industrial process-mining goals [28] also makes it necessary to align the available evidence with the intended analytical claims.

Building on these methods, we study the construction of shared event representations from egocentric factory recordings. The emphasis is on making the path from observations to process measurements inspectable, so that the resulting descriptions of work can be interpreted in light of their construction.

3. Problem Definition

The starting point is the factory video in the Egocentric-10K dataset. A camera worn by a worker records visible actions involving tools, materials, and work surfaces. These recordings contain evidence of work, but they do not directly identify the interval events needed for process mining. We therefore first use a vision-language model to produce a generated transcript: timestamped natural-language descriptions of visible actions in each video. We then use large language models (LLMs) to organize the descriptions into shared factory vocabularies and convert the generated transcripts into interval events. Each event should state what action occurred, its broader process context, and its estimated start and end within the recording.

Let F be the set of factories, W_f the worker identifiers associated with factory f, and V_w the recordings associated with worker w. Each video v has an underlying duration D_v, which need not be available in the derived archive. Its generated transcript T_v is an ordered collection of entries (s_i, e_i, x_i), where x_i describes the interval from s_i to e_i. The descriptions and endpoints are model outputs, so the generated transcript is an intermediate representation of the video rather than an independent annotation.

For each factory, the pipeline induces an activity vocabulary A_f and a process vocabulary P_f. Each extracted event records the source recording, worker, factory, estimated start

Table 5. Notation and intended meaning.

Symbol	Meaning
f, w, v	Factory, worker, and video identifiers; identifiers do not certify roles or task equivalence
D_v, T_v	Source duration and generated transcript; the former is unavailable in the released subset
A_f, P_f	Two separate permitted label lists for factory f
t_s, t_e, d_e	Estimated video-relative start, end, and duration $d_e = t_e - t_s$, in seconds
$\kappa(e)$	Selected case identifier, such as the source video
$g_f(p)$	Analyst's mapping from process label to operational category
C_v, S_v	Interval-union event duration and endpoint span used in coverage diagnostics

and end times, activity label, and broader process context. The following tuple gives a compact description of one extracted episode in that order:

$$e = (v, w, f, t_s, t_e, a, p), \qquad a \in A_f, \quad p \in P_f. \tag{1}$$

These source links provide provenance, meaning information about where an event and its description came from. The estimated event endpoints t_s, t_e are relative to the video. The generated transcript endpoints delimit descriptions; the extracted event endpoints delimit the events produced from those descriptions. One description can therefore become several events with different boundaries. The LLM interprets T_v using A_f, P_f, and the worker context to produce the event set E_v; the complete log contains $E = \bigcup_v E_v$.

Process analysis also requires a rule for deciding which events belong to the same sequence, or trace. The case function $\kappa : E \to K$ assigns each event to one of these sequences, where K is the set of case identifiers. For example, grouping events by their source video makes each recording a case. Section 5.4 compares the available case choices. The construction problem is thus to transform videos into generated transcripts and then into labeled intervals, while retaining enough source information to inspect the resulting log. Vocabulary induction uses descriptions across the available factory recordings, so event extraction depends on both the local generated transcript and its factory context.

Table 5 summarizes the notation. An event represents an operational episode. One episode may summarize many repeated actions, and several events may divide a single narrated interval.

3.1. Activity, process context, and analytical category

An *activity* names the action; its *process context* names the broader operation. An *operational category* groups events for analysis. The extractor checks $a \in A_f$ and $p \in P_f$ separately, without enforcing compatibility or a hierarchy between the labels.

We assign categories using the process label, $g_f(p)$. Value-adding (VA) covers production and completion work under the authors' scheme; it does not directly measure customer value. Non-VA includes all other categories. A transport activity can therefore

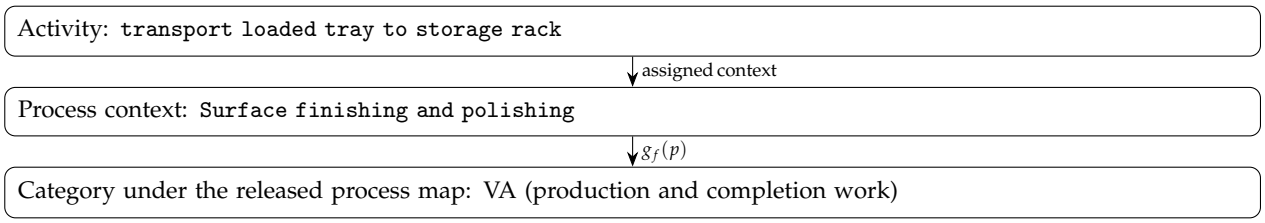


Figure 3. Transport counted as VA through its process label. Source: factory **003**, worker 001, recording 00001, event row 2, 00:03:52 to 00:04:00 (8 seconds).

Table 6. Conceptual tasks and their roles in the five-stage construction. The stages interact; this is not a claim that each task has an isolated error rate.

Task	Main stage	Decision made
Temporal segmentation	**S1**	Describe candidate episodes with estimated boundaries; merge stable repetition
Activity-class construction	**S2** and **S3**	Aggregate worker descriptions and induce reusable factory activity and process lists
Event-instance assignment	**S4**	Select permitted labels for episodes; retain or split generated transcript intervals and estimate internal boundaries

Table 7. Stage specification from scripts 01 to 05. A nonempty text output is not a validation of its factual content.

Stage; unit	Inputs and context	Output	Failure and reuse behavior
S1; video	Uploaded video; transcription prompt	One chronological text file	Keeps existing files; retries transient failures; rejects an empty response; attempts deletion of uploads
S2; worker	All saved generated transcripts in one request; filenames omitted from the bundle	Worker report	Missing or empty input raises an error; empty response rejected; keeps existing output unless overwrite is set
S3; factory	All worker summaries in one request, with worker identifiers	Factory report with two label lists and compact worker entries	Same text and reuse checks as **S2**; malformed section structure may fail later in **S4**
S4; generated transcript	Generated transcript; two label lists; compact **S3** worker entry, or a missing-context marker	Per-video CSV	Missing lists or invalid events raise errors; an empty event list is valid; existing CSV kept unless overwrite is set
S5; worker, then factory	CSVs in sorted filename order	Worker and factory CSVs	Checks timestamp columns and consistent headers; empty files add no time; keeps existing output if its schema matches

contribute to VA time through its process label (Figure 3). Categorizing by activity instead would change the measurement.

4. Pipeline and Design Rationale

The released scripts implement five stages. **S1** describes the visible actions in each video as a generated transcript. **S2** summarizes each worker's generated transcripts. **S3** combines the worker summaries into a factory report and shared label lists. **S4** extracts events from each original generated transcript using those lists and the compact worker entry from the **S3** report. **S5** merges the resulting CSVs. Figure 2 in the Introduction gives an overview of these stages. Reports and logs share evidence, so their agreement is not independent validation.

```
for each selected factory f:
    for each available video v:
        T[v] = reuse existing transcript or S1(video[v], prompt1)
    for each worker w:
        B = concatenate all T[v] for w, without filenames
        R[w] = reuse existing summary or S2(B, prompt2)
    F[f] = reuse existing report or S3(all R[w] for f, prompt3)
    A, P, worker_context = parse the required sections of F[f]
    for each transcript T[v]:
        context = T[v] + A + P + worker_context[w]
        E[v] = reuse CSV or normalize(S4(context, prompt4))
    for each worker w:
        offset = 0
        for each CSV v in sorted filename order:
            append E[v] shifted by offset; keep local offsets
            if E[v] is nonempty: offset += max end(E[v])
    concatenate worker logs into the factory log
```

Listing 1: Algorithm 1: implemented dependency order. API retries surround requests; this pseudocode does not add windowing or new validation.

Adaptation takes place in the context and vocabulary, while the foundation-model parameters remain fixed. Each available factory corpus contributes to its own label lists through **S2** and **S3**, and **S4** then uses those lists to label recordings from that corpus. The six-factory assessment therefore applies the same construction procedure to six separately induced representations. It is a retrospective use of the available material, not an evaluation of a fixed vocabulary on a previously unseen factory. Prompts also supply analyst choices about operational relevance and granularity.

4.1. Vocabulary construction as event abstraction

Event abstraction concerns the level at which observations represent process behavior [20]. Table 6 separates three decisions in our implementation. Constructing a reusable class and assigning an event to it are different tasks: a missing concept and a mistaken assignment require different checks. The process list supplies a second label dimension, with no enforced parent-child relation to the activity list.

4.2. Execution units and failure behavior

Table 7 states the input, output, and failure behavior for each execution unit. Listing 1 gives their dependency order. Listings 2 to 5 reproduce the four full prompt templates, with punctuation normalized for display. The original templates remain in the resource. Placeholder braces are filled by the scripts; the additional context is specified in Table 7. The prompts specify requested model behavior, not guaranteed output properties.

```
Please transcribe the attached egocentric video into a very detailed chronological text description.

I do not want a formal event log, structured schema, activity labels, or any kind of post-processing. I only want plain text entries, each
beginning with a timestamp range, followed by a detailed description of what happens in that time span.

Requirements:
- Format every entry as:
  [HH:MM:SS - HH:MM:SS] description
- Follow the video strictly from start to end.
- Use short timestamp intervals whenever something changes.
- Be highly detailed, but do not repeat the same type of behavior again and again when nothing changes.
- If the same action continues in the same way, with the same context and no relevant difference, merge it into one longer timestamped entry
 instead of splitting it into many repetitive entries.
- Only create a new entry when something meaningful changes, such as:
  - the action changes
  - a new object is handled
  - the person changes location
  - attention shifts to something else
  - another person becomes involved
  - something unusual, problematic, interrupted, or different happens
- Capture micro-actions when visible, such as reaching, grasping, lifting, placing, opening, closing, turning, walking, pausing, looking,
adjusting, and interacting with objects or people.
- Describe what is visible in the egocentric view, including hand movements, object handling, scene changes, and shifts of attention.
- If a repeated behavior continues in exactly the same way, summarize that repeated stretch once with an appropriate timestamp range instead
 of listing each repetition separately.
- If the scene is partially unclear, mention the uncertainty briefly instead of guessing.
- Do not use tables, bullets, JSON, labels, categories, or analysis language.
- Do not convert the result into an event log.
- Do not add any summary at the end.
- Only provide the timestamped descriptive transcription.

The output should read like a detailed time-aligned narration of the video, not like a structured annotation file.

Example of the intended compression:
- Good:
  [00:03:10 - 00:03:42] The wearer continues wiping the same section of the table with repeated back and forth hand movements. No new object
 or interruption appears.
- Not desired:
  [00:03:10 - 00:03:14] The wearer wipes the table.
  [00:03:14 - 00:03:18] The wearer continues wiping the table.
  [00:03:18 - 00:03:22] The wearer continues wiping the table.

Please produce the full transcription in this style for the entire video.
```

Listing 2. Full video transcription prompt for **S1**.

```
You are given multiple detailed chronological transcriptions from egocentric factory videos recorded by the same worker.

Your task is to synthesize those transcriptions into a process-focused report about what work is being done and what process-level
optimization opportunities are visible.

Context:
- Factory identifier: {factory}
- Worker identifier: {worker}

Important output constraints:
- Do not mention the factory identifier, worker identifier, file names, file paths, transcript numbers, video numbers, or any other source
identifiers.
- Do not mention timestamps unless they are absolutely necessary, which they normally are not.
- Do not quote the transcripts.
- Do not write a per-video recap. Synthesize across all provided transcriptions.

Required focus:
- Identify the process or processes performed by the worker.
- If the worker appears to perform more than one distinct process, separate them clearly.
- Describe the operational flow in enough detail that someone can understand how the work is carried out.
- Include relevant details such as setup, repeated action loops, machine interaction, material handling, part movement, quality checks,
cleaning, waiting, transport, handoffs, and interaction with nearby stations when visible.
- Stay grounded in the provided evidence. If something is uncertain, state the uncertainty briefly instead of inventing details.

Optimization constraints:
- Only suggest process optimizations that are visible or reasonably inferable from the observed workflow.
- Focus on process design, workstation layout, batching, staging, transport, sequencing, buffering, machine idle time, tool/material
presentation, inspection placement, handoffs, or coordination between steps.
- Do not suggest worker-level optimizations such as "work faster", "be more careful", "improve discipline", "train the worker better", or
similar people-focused advice.
- If no credible process optimization is evident, say so.
- When an optimization is inferential rather than directly observed, label it clearly as an inference.

Write the output in plain text with exactly these section headings:

Process Summary
Detailed Process Breakdown
Possible Process Optimizations
Evidence Limits
```

Listing 3. Full worker-summary prompt for **S2**.

The **S1** prompt (Listing 2) asks for a new timestamped entry when an action, object, location, or interaction changes. It asks for repeated stable work to be summarized once and for uncertainty to be stated briefly. These are instructions to the model. **S1** does not check whether each visible change was narrated or whether the timestamps match the video.

The **S2** worker-summary prompt (Listing 3) and the **S3** factory-report prompt (Listing 4) compress large text bundles into summaries. This makes factory-wide vocabulary induction possible, but uncommon actions can be lost. The implementation does not automatically split long requests into windows, reconcile separate summaries, or check coverage against all generated transcript spans. **S4** partly bypasses this compression by reading each original generated transcript, although it must still use the induced vocabularies.

The **S4** event-extraction prompt (Listing 5) requests a strict JSON schema with estimated start and end times, activity, and process. Its local parser can also accept a list, strip

Table 8. Supervision and human input in this study. Unlike labeled threshold calibration in Kecht et al. [10] or parameter adaptation in SnapLog and Seeth et al. [11,12], generation here adapts through prompts and induced context.

Dimension	Role in the present procedure
Model pretraining	General-purpose pretrained models provide visual and language capabilities
Task-specific parameter updates	None in **S1** to **S4**; no model is trained or fine-tuned on these factory recordings
Labeled examples	No labeled event examples for training, prompting, or calibration; **S1** includes an illustrative narration-format example
Vocabulary origin	**S3** induces two lists from the available worker summaries; **S4** checks list membership
Domain and analyst input	Authored prompts define relevance and segmentation; analysts choose case projections and the operational-category map
Human assessment	One author's archived event and transcript-entry judgments assess outputs; they are not generation inputs

```
You are given multiple worker-level process summaries from the same factory.

Your task has two goals:
1. Produce a factory-level report that restates the worker process summaries in a compact but informative way, preserving which worker each
summary belongs to.
2. Produce a long, detailed, process-mining-oriented activity catalogue for the factory that can help with downstream process optimization.

Context:
- Factory identifier: {factory}

Important constraints:
- Do not mention file names, file paths, source identifiers, transcript numbers, or video numbers.
- Do not quote the source summaries.
- Stay grounded in the provided evidence. If something is uncertain, state the uncertainty briefly instead of inventing details.
- Synthesize across workers. Do not merely copy the worker summaries back verbatim.

Worker summary requirements:
- Include every worker provided in the input.
- Keep each worker summary concise but useful.
- Preserve the worker identifier for each summary, because the output is factory-level.
- Focus on the main process(es), major material flows, handoffs, machine interactions, transport, inspection, waiting, cleaning, batching,
and coordination patterns that matter operationally.

Factory process label requirements:
- Explicitly provide one raw deduplicated list of processes executed across the different workers in the factory.
- Express each process as a short label only.
- Keep process labels concise and comparable across the factory.
- Prefer labels such as "metal stamping", "box assembly", "garment bagging", "part transport", "inspection", or "machine cleaning".
- Distinguish clearly between materially different processes, but do not create unnecessary micro-variants.
- Do not group the labels by worker.
- Do not repeat the same label for multiple workers.

Activity catalogue requirements:
- The catalogue must be long and detailed.
- Each activity must be a short string only, suitable as a candidate event label for process mining.
- Prefer specific verb-object phrases such as "load raw blank into press" rather than generic labels such as "operate machine".
- Include not only value-adding operations, but also transport, staging, waiting, inspection, rework-like handling, setup, cleanup, handoff,
 documentation, and coordination activities when visible.
- Keep distinct activity variants separate when they matter for process analysis.
- Avoid duplicate or near-duplicate labels.
- Avoid full sentences, explanations, numbering inside the activity text, or extra commentary next to the activity.
- If an activity is not clearly supported, do not invent it.

Optimization relevance requirements:
- Favor activity labels that would help reveal bottlenecks, queues, machine idle time, material handling burden, unnecessary motion,
batching behavior, rework, inspection points, coordination dependencies, and handoff structure.

Write the output in plain text with exactly these section headings:

Factory Process Overview
Worker Process Summaries
Factory Process Labels
Factory Activity Catalogue
Optimization-Relevant Process Observations
Evidence Limits

Formatting rules for the output:
- Under "Worker Process Summaries", use one line per worker in the form:
  worker_xxx: summary
- Under "Factory Process Labels", use one bullet per process label.
- Each bullet in "Factory Process Labels" must contain only a short process label and nothing else.
- Under "Factory Activity Catalogue", use one bullet per activity.
- Each bullet in "Factory Activity Catalogue" must contain only a short activity string and nothing else.
```

Listing 4. Full factory-report and vocabulary prompt for **S3**.

code fences, recognize fallback time keys, and parse two-component offsets or numeric seconds. It normalizes label case and surrounding quotes, checks membership, rejects end-before-start intervals, and sorts rows by start, end, activity, and process.

S1 to **S3** default to `gemini-3.1-pro-preview`; **S4** defaults to `gpt-5.4` and requests high reasoning effort. Appendix B of the Additional Material specifies the distinction between code defaults and historical execution evidence.

4.3. Requests, guarantees, observations, and expectations

"Automated extraction" means that generation uses no task-specific fine-tuning or labeled event examples. It does not mean that the complete study is free of human judgment.

```
You are converting one timestamped raw factory annotation into a structured event list for process mining.

Context:
- Factory identifier: {factory}
- Worker identifier: {worker}

You will be given:
- the allowed process labels for the factory
- the allowed activity labels for the factory
- an optional worker summary for additional context
- one raw annotation transcript with timestamp ranges measured from the start of the video

Your job:
- Convert the raw annotation into a JSON object with one key, ‘events‘, whose value is a list of event objects.
- Each event must correspond to one operationally meaningful event segment.
- Use the transcript timestamps as the main basis for the event boundaries.
- If a transcript interval contains one repeated stable activity, keep it as one longer event.
- If a transcript interval clearly contains multiple different events, you may split it into multiple estimated events.
- Keep the events in chronological order.

Strict label constraints:
- The value of ‘activity‘ must be exactly one label from the allowed factory activity list.
- The value of ‘process‘ must be exactly one label from the allowed factory process list.
- Do not invent new activity labels.
- Do not invent new process labels.
- If multiple labels are plausible, choose the closest allowed label.
- Be especially careful to respect these two controlled vocabularies exactly.

Output schema:
- Return JSON only.
- Return a top-level JSON object with exactly one key: ‘events‘.
- The value of ‘events‘ must be a JSON array.
- Each array item must be an object with exactly these keys:
  - ‘estimated_start_time‘
  - ‘estimated_end_time‘
  - ‘activity‘
  - ‘process‘
- ‘estimated_start_time‘ and ‘estimated_end_time‘ must be strings in ‘HH:MM:SS‘ format, measured from the start of the video.

Additional rules:
- Include only process-relevant operational events. Ignore purely accidental camera motion or camera removal unless it directly reflects a
process-relevant activity.
- Do not output markdown fences.
- Do not output explanations, notes, comments, or any text before or after the JSON object.
- If no valid operational events can be extracted, return ‘{"events":[]}‘.
```

Listing 5. Full event-extraction prompt for **S4**.

Table 8 identifies the different inputs. “Constrained” refers to the separate permitted label lists. “Validated” is used only with a named check, never as a general claim of visual accuracy.

4.4. Design choices and tradeoffs

Table 9 summarizes the design choices and alternatives. Repeated cycles deserve particular care. If ten cycles become one narrated interval and one event, the event count decreases and the directly-follows relation loses nine possible repetitions. A longer event

Table 9. Design rationale and conceptual alternatives. No alternative in the last column was experimentally compared here.

Choice	Intended benefit	Limitation	Possible alternative
Video-to-text intermediate	Readable evidence; reuse for different analyses	Visual errors become shared textual evidence	Direct video-to-events, with stronger visual provenance
Worker summaries	Compact factory-level context	Rare behavior and recording identity may be lost	Window summaries with explicit source references
Factory vocabularies	Concrete local labels across workers	Different factories use different abstractions	Global concepts with local extensions
Closed labels	Prevent out-of-list output	Closest allowed label may still be wrong	Open labels followed by review and reconciliation
Change-point narration	Compact description of stable behavior	Boundary detail and repeated cycles can disappear	Finer segments with explicit repetition counts
Interval events	Preserve approximate episode duration	Row counts do not equal production volume	Object-linked cycle records, when supported

can represent more repetitions, slower work, or more aggressive compression. The logs alone do not distinguish these explanations. The text interface makes this decision inspectable, but does not recover details omitted from the generated transcript.

5. Dataset and Released Artifacts

5.1. An inferred resource for process analysis

The resource is available at https://github.com/fit-alessandro-berti/annotated-egocentric-10k-dataset. It links generated transcripts, worker summaries, factory vocabularies, inferred events, logs, and analysis artifacts from six factories. Researchers can inspect how events are constructed, compare alternative representations, and examine how activity and process labels affect measurements.

Assembly101 [29] provides toy assembly and disassembly recordings from static and egocentric cameras, with action annotations for recognition, anticipation, and segmentation. The present resource focuses on the construction of inferred events and their use in process analysis.

The labels and boundaries are inferred by the pipeline. Assessment includes structural checks and an author audit against generated transcripts, without independent video annotations. Comparisons therefore evaluate representation choices unless checked against video; the resource is not a ground-truth benchmark for action recognition or boundary detection.

5.2. Selection and provenance

The archived subset contains outputs for factories **001** to **006**, with 59 worker identifiers and 322 paired generated transcript and event files. Factory and worker identifiers are inherited from the source organization. Domain descriptions in this paper summarize generated factory reports; they are not independently verified factory metadata. The archive does not record the selection rationale, the original source revision, or a complete inventory of the recordings available at extraction time. We therefore describe an available processed subset, not a random or representative sample of factories.

The current provider card reports a median clip length of 180 seconds [3], whereas many archived generated transcript endpoints lie near 20 minutes. These describe different quantities and possibly different snapshots or subsets. The card documents a per-video `duration_sec` field inside source archives, but those metadata files are not included here. The pipeline scans video files and does not preserve that field. The selected recordings' actual durations therefore cannot be established from the derived archive. The inspected public dataset snapshot does not identify the historical source version used for extraction.

Table 10. Artifact inventory. Counts come from the frozen repository outputs. Worker and factory merged logs repeat the same source events.

Layer	Files	Content and provenance
Generated transcripts	322	One text per video; recording identity in path
Worker summaries	59	One text per worker; combines generated transcripts
Factory reports	6	Narrative, worker entries, activity and process lists
Per-video logs	322	10,698 interval-event rows; six stored columns
Merged worker logs	59	Same 10,698 events with source file, worker case, and local offsets
Merged factory logs	6	Same 10,698 events concatenated across workers
Audit record	1	42 event-support judgments and 42 transcript-entry-overlap judgments
Prompts; stage scripts	4; 5	Extraction instructions and implementation

The six per-video columns are `start_timestamp`, `end_timestamp`, `activity`, `process`, `factory`, and `worker`. **S5** adds `source_file`, `case_id`, video-relative start and end offsets, and duration. Table 10 distinguishes file counts from unique event counts. The released dataset documentation follows the documentation concerns of Datasheets for Datasets [30]: composition, collection, uses, and limitations.

There are 13,304 generated transcript entries recognized by the original three-component timestamp parser. Recognizing the additional two-component and mixed-format entries gives 13,462 entries, without rescaling any timestamp.

Across all logs, the original endpoint-span calculation totals 786,412 seconds (218.4 hours). This sums parsed spans, not verified footage durations. Interval unions cover 414,886 seconds by counting overlapping time only once. Summed event durations total 421,283 seconds because each row contributes its full duration. Their difference, 6,397 seconds, is the additional time counted through overlaps.

The retained time-analysis subset contains 10,647 events in 320 logs, with 349,659 seconds of summed event duration (97.1 hours). This is the duration used for category shares; it is neither the full archive's endpoint span nor its interval-union duration.

5.3. Timestamp formats and inclusion rules

The time analysis excludes a log if any event ends more than one hour after the start of its recording. This rule uses event times, so an unusual timestamp in a generated transcript does not automatically exclude its log. It removes two logs, both from factory **001**, and keeps the other five in Table 11. Keeping a log does not confirm that its timestamps are correct.

Table 11 compares the latest end times in the seven generated transcripts that exceed one hour when read as hours:minutes:seconds. For example, one transcript ends at `20:06:00`, while its event log ends at `00:20:06`. This may reflect a timestamp-format error, but the available data do not establish which time is correct. We keep the recorded timestamps without correcting or rescaling them.

Timestamp formats also affect how many transcript entries can be read. The original parser accepts only `HH:MM:SS` (hours:minutes:seconds). An additional check also accepts `MM:SS` (minutes:seconds) and files that mix both formats. Four other transcripts contain `MM:SS` times: **001**/004/00001, **002**/005/00001, **002**/009/00003, and **003**/003/00003. Three mix formats; **002**/005/00001 uses `MM:SS` throughout. The original parser misses 158 entries in these files, explaining the different entry counts reported above.

These timestamp inconsistencies affect the coverage ratios in Section 7.2. A low ratio alone does not show that extraction missed later parts of a long video.

Table 11. Latest end times in seven generated transcripts and their event logs. All times are shown as hours:minutes:seconds. The transcript times exceed one hour, but a log is excluded from the time analysis only if an event end time exceeds one hour. IDs indicate factory/worker/video. Actual video durations are unknown.

ID	Generated transcript endpoint	Event endpoint	Time-analysis decision
001/001/00004	20:00:00	20:00:00	Exclude
001/010/00001	01:20:00	01:20:00	Exclude
002/002/00008	20:00:00	00:20:00	Retain
002/010/00001	20:00:00	00:20:00	Retain
005/002/00000	13:52:00	00:13:52	Retain
006/001/00001	20:06:00	00:20:06	Retain
006/004/00000	19:59:00	00:19:59	Retain

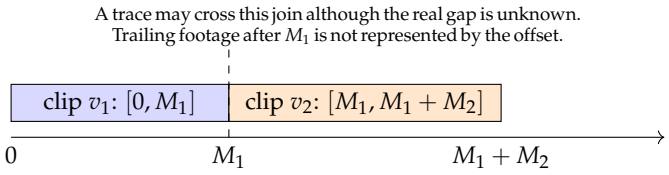


Figure 4. Schematic of the implemented merge, not measured source time. The first clip occupies the synthetic span from zero to its last extracted endpoint; the second starts there and adds its own endpoint span. Separate video cases preserve the clip boundary. A worker case creates a directly-follows relation across it.

5.4. Synthetic time and case selection

Each recording has its own local timeline. To combine recordings into a worker sequence, the merge shifts their event times onto a common artificial timeline. **S4** stores local offsets as dates relative to 2000-01-01 for software compatibility. The archive provides no verified calendar times or cross-worker synchronization. **S5** processes clips in filename order, which is not proof of their real chronological order.

For a worker's ordered clips $v_1, \ldots, v_m$, the last extracted event endpoint in recording j is $M_j = \max_{e \in E_{v_j}} t_e$, with $M_j = 0$ for an empty log. Let O_j be the accumulated time shift for that recording, and let the primed endpoints be the shifted event times. **S5** adds the preceding recording's endpoint to the accumulated shift and applies it to each event:

$$O_1 = 0, \qquad O_{j+1} = O_j + M_j, \qquad (t'_s, t'_e) = (O_j + t_s, O_j + t_e). \tag{2}$$

The same shift is added to both endpoints, preserving the event's duration. For example, if the first recording's last extracted endpoint is 60 seconds, an event at 10 to 20 seconds in the next recording appears at 70 to 80 seconds. The next clip starts at the largest extracted endpoint, not at the full source duration. Unrepresented trailing footage and empty logs add no elapsed time. Within-clip gaps before that endpoint remain on the synthetic axis, but gaps between recording sessions are unknown. Each worker has a separate synthetic origin; equal synthetic dates do not establish simultaneous work.
Figure 4 illustrates the synthetic clip join, and Table 12 compares the case rules supported by the released attributes.

6. Assessment Methodology

The assessment examines the structure of the extracted events, their consistency with generated descriptions, and their use in the reported descriptive analyses. It combines structural checks, the existing author audit, and calculations with explicitly defined measures.

The inputs are frozen texts, reports, CSVs, and archived author judgments. The descriptive workflow runs offline with Python's standard library. Activity discovery uses

Table 12. Case constructions supported by the attributes. Process episodes require an additional grouping step and are not stored as case IDs.

Case rule	Supported description	Interpretation risk
Video as case, $\kappa(e) = v$	Local activity order within an observation window; used for discovery	Window may start or end midway through a production process
Worker as case, $\kappa(e) = (f, w)$	Sequence of retained clips; stored **S5** case ID	Filename joins can create artificial transitions; calendar duration is unknown
Contiguous process episode	Consecutive events with the same process within a chosen clip, with a stated gap rule	Inferred context boundaries do not identify items or completed instances

Table 13. Evaluation questions, required references, and evidence in this study. Successful computation and correct operational interpretation are separate claims.

Question	Reference needed	Present evidence and scope
Are visual actions correct?	Independent annotations of the video	Not measured; generated transcripts cannot detect their own visual errors
Are labels consistent with text?	A reference interpretation of the text	Existing author judgments for 42 events; label membership checked for all rows
Are boundaries accurate?	Independently defined action boundaries	Agreement with generated transcript endpoints only; the five-second tolerance is a matching rule
Can the log support process-mining computations?	Defined case rules, measures, and completed calculations	Discovery and descriptive outputs demonstrate computation on the inferred representation
Are operational conclusions correct?	Valid measurements and evidence relevant to the conclusion	Traceable category descriptions and hypotheses; no independent production outcomes or intervention evidence

PM4Py [31] with the archived miner and noise-threshold settings. All calculations use the released records as their input.

6.1. Evaluation questions and reference data

Table 13 distinguishes the objects of evaluation. Our generated transcript checks assess an intermediate transformation, while our downstream calculations demonstrate uses of its output. These assessments support different conclusions, so their accuracy, fitness, and precision values should not be ranked as if they measured the same property.

6.2. Metrics, denominators, and aggregation

Table 14 defines each measure, its denominator, and how recordings or workers contribute to the result.

Coverage is the fraction of the parsed timestamp span occupied by extracted events. For video v, let E_v be its events, B_v the transcript endpoints recognized by the original parser, $T_v^{\max}$ their maximum, and M_v the latest event endpoint. Factory coverage is:

$$C_v = \left| \bigcup_{e \in E_v} [t_s, t_e) \right|, \qquad S_v = \max(T_v^{\max}, M_v), \qquad \text{coverage}_f = \frac{\sum_{v \in f} C_v}{\sum_{v \in f} S_v}. \tag{3}$$

C_v is occupied time in seconds, with overlaps counted once; the vertical bars denote interval duration. S_v runs from zero to the latest transcript or event endpoint. Dividing factory totals weights recordings by this span. If no transcript endpoint is recognized, M_v supplies the denominator. These spans come from recorded timestamps; actual video durations are unavailable. We retain the original parser's results and release the broader parser's results separately.

Boundary agreement is the fraction of event starts and ends within five seconds of a recognized transcript endpoint. Each event contributes two boundaries. If a recording has no recognized endpoints, its boundaries count as unmatched. Several boundaries may match one endpoint. This measures timestamp agreement without establishing action accuracy or validating an internal split.

The process map assigns events to *value-adding production work* (VA), *material handling and staging* (MH), *transport* (TR), *quality inspection* (QI), *cleaning and maintenance* (CM), *rework* (RW), and *documentation* (DOC). VA includes packaging and completion work under the authors' convention; it does not directly measure customer value. Defect patching is RW because its descriptions refer to surface repairs, without establishing avoidability or failure against a quality standard. Work outside VA may still be necessary.

Table 14. Metric definitions. E_f^* is the retained set under $M_v \leq 3600$; E_w^* is its worker subset. Summed event-hours are $H_w = \sum_{e \in E_w^*} d_e / 3600$. R_w^{VA} is the worker's list of VA run durations, converted to minutes. Boundaries in recordings with no recognized generated transcript endpoints count as unmatched.

Quantity	Definition	Unit, aggregation, exclusions
Vocabulary membership	$\#\{e : a \in A_f,\ p \in P_f\} / \#E_f$	Fraction of all event rows
Vocabulary use	Distinct assigned labels divided by permitted labels	Separate activity and process lists, per factory
Coverage diagnostic	$\sum C_v / \sum S_v$	Fraction of parsed endpoint span; all logs
Boundary agreement	$\sum_{e,b \in \{t_s,t_e\}} \mathbf{1}[\min_{t \in B_v} \lvert b - t \rvert \leq 5] / (2\#E)$	Fraction of boundaries; all logs; original parser
Category share	$q_{fc} = \sum_{e \in E_f^*} d_e \mathbf{1}[g_f(p_e) = c] / \sum_{e \in E_f^*} d_e$	Fraction of summed event duration; overlap counted in each row
Worker VA share	Same ratio restricted to E_w^* and category VA	Eligible workers require $H_w \geq 0.5$; factory median weights workers equally
VA transition rate	I_w / H_w, where I_w counts VA-to-non-VA transitions in concatenated run lists	Transitions per summed event-hour; factory median
Mean VA run	$\overline{R}_w = \sum R_w^{VA} / \#R_w^{VA}$	Minutes; table reports median across eligible workers of these means
MH/TR count and rate	Number of retained MH or TR rows; count divided by summed event-hours	Rows, not merged episodes; duration-weighted factory denominator
Dominant process	$\arg\max_p \sum_{e \in E_w^*: p_e = p} d_e$	Groups workers; comparisons then use all their retained events
Self-loop share	Equal adjacent activity labels divided by all adjacent pairs	Within video cases only; all logs

Of the 113 factory-specific process entries (108 distinct strings), 112 are used and mapped. The unused factory **005** label *motor assembly finishing* has no category assignment. Appendix D of the Additional Material lists the full map.

Category shares divide a category's summed event duration by that of all retained events. Retention requires $M_v \leq 3600$; E_f^* and E_w^* denote the retained factory and worker events. The process label determines the category, and each row contributes its full duration, including overlaps.

Worker summaries use the same retained events. Worker event-hours H_w are summed durations divided by 3600. Factory medians give eligible workers equal weight. The *dominant process* has the largest summed duration and determines grouping; each worker's statistics still use all retained events. In Table 14, # counts elements, **1** is one when its condition holds and zero otherwise, and arg max selects the process with the greatest summed duration.

Runs group consecutive events of the same category within a recording. Their durations sum event durations, so gaps may be bridged and overlaps counted. We calculate each eligible worker's mean VA-run duration in minutes, then report the factory median. No eligible worker has an empty VA-run list; the implementation would return zero for that case.

VA transition rates count changes from VA to another category in concatenated worker run sequences and divide by summed event-hours. This includes 15 transitions across clip joins among eligible workers; the counts do not establish interruptions in continuous work. As in the original script, worker VA shares, transition rates, and mean VA-run durations are rounded to one decimal before factory medians are calculated.

All per-video logs are nonempty, with no negative or zero durations. Overlaps count in duration totals and are merged only for coverage. Ambiguous transcript timestamps are used as recorded.

6.3. Existing semantic audit protocol

The audit compares extracted events with generated text using two samples: 42 events and 42 transcript entries, seven of each per factory. One author judged both samples. The record gives a random seed of 42 and sampling by factory, but not the full procedure,

candidate pool, selection order, or treatment of timestamp formats. The sampling cannot be repeated exactly from these records.

For *event support*, C means the overlapping transcript fully supports the label, P means the label is plausible but not fully supported, and X means contradiction. The sample contains 36 C events, six P events, and no X events. Strict support is $36/42 = 0.857$, with partial cases kept in the denominator.

For *transcript-entry overlap*, COV means an entry overlaps at least one event, MISS means an uncovered entry judged operational, and EXC means an uncovered entry judged non-operational. There are 34 COV entries, four MISS entries, and four EXC entries. Keeping MISS and excluding EXC yields $34/(34+4) = 0.895$, the *reported entry-overlap rate*. It does not measure action recall: one COV entry describes camera tilt covered by a longer event, and the 38 retained entries are not independently verified as operational. Overlap alone does not establish complete or correct extraction of actions and labels.

The audit measures agreement with generated transcripts and cannot detect errors already present in them. There is no independent evaluator, inter-rater agreement estimate, or video audit. The sample rates do not bound errors in time shares, categories, observation selection, or causal interpretation. Appendix E of the Additional Material reproduces every judgment and note, including the flagged, unverified truncation explanation.

7. Results and Factory Case Studies

7.1. ***RQ1***: *Construction and structural properties*

Table 15 reports the released corpus. All 10,698 events have the required values, nonnegative ordered endpoints, and labels in the relevant factory lists. Start times are nondecreasing within each log. There are no empty logs or zero-duration events. However, 35 adjacent event pairs overlap, 20 in factory **001** and 15 in factory **002**. Passing schema and vocabulary checks therefore does not imply a non-overlapping segmentation. Activity vocabulary use ranges from 89.3% to 96.7%.

7.2. ***RQ2***: *Consistency with generated transcripts*

The original parser finds 82.1% of event boundaries within five seconds of a recognized generated transcript endpoint. It yields an overall coverage diagnostic of 52.8%. Neither statistic is video accuracy. The coverage denominator is heavily affected by the literal multi-hour endpoints listed in Table 11. Boundary agreement can also be low when formats change or when **S4** splits a described interval. These diagnostics alone do not establish extraction truncation.

Table 15. Corpus and generated transcript diagnostics using the original parser. Activity and process columns show permitted/used labels. Coverage divides occupied event seconds, counting overlaps once, by parsed endpoint spans rather than verified video duration. Boundary agreement uses the five-second matching rule under the original parser.

Factory	Workers	Videos	Events	Activities	Processes	Coverage	Boundary agreement
001	11	97	2,288	131/125	13/13	94.8%	93.1%
002	10	54	1,541	75/67	24/24	28.2%	71.8%
003	11	72	2,449	55/52	12/12	93.0%	86.8%
004	10	51	1,893	71/65	22/22	92.1%	90.4%
005	11	22	1,103	102/97	18/17	25.0%	75.0%
006	6	26	1,424	61/59	24/24	14.1%	62.1%
All	59	322	10,698			52.8%	82.1%

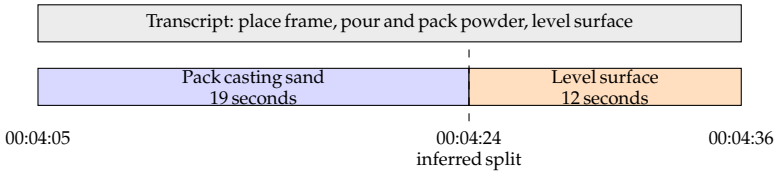


Figure 5. Recorded transcript interval and inferred event intervals for the foundry example. The generated transcript entry is line 16 of its text file; the events are data rows 17 and 18 of the corresponding CSV. Bar lengths show recorded durations, not independently checked action boundaries.

Table 16 summarizes the archived judgments. Partial-support examples show the difficulty of checking events against compressed text. Event sample 5 assigns a particular bagging activity where the generated transcript only states that repetitive assembly and packaging continue. Sample 40 assigns guiding an upper mold half where the text only says that the worker helps with a mold. Transcript-entry sample 10 uses an interval from 11:51:00 to 12:59:00 in an anomalous generated transcript whose event log ends at 00:20:00. Its MISS judgment is preserved, but the original explanation as long-recording truncation is unverified. These cases show why a single accuracy label would obscure different sources of uncertainty.

7.3. An end-to-end foundry example

This example shows how a description of several actions becomes two labeled events with an inferred internal boundary. In factory **006**, worker 003, recording 00003, the generated transcript from **S1** describes actions from 00:04:05 to 00:04:36: placing an empty frame on a pattern plate, pouring and hand-packing powder, and using a wooden tool to level the surface. The worker summary from **S2** provides context by describing preparation, filling, packing, leveling, and compression of molds across recordings. The factory vocabulary from **S3** supplies process *Sand mold creation* and activity *level powder surface with wooden scraper*. The **S3** report also supplies a compact summary of this worker's molding work.

Event extraction in **S4** creates two intervals within that 31-second generated transcript entry: *pack casting sand into mold frame*, from 00:04:05 to 00:04:24, and *level powder surface with wooden scraper*, from 00:04:24 to 00:04:36. Both have process *Sand mold creation*. Figure 5 shows what is retained and added. The internal boundary at 00:04:24 is inferred; **S1** does not timestamp that transition separately. The events compress placement, pouring, and joint hand-packing into the first activity and add shared labels and a process context.

The linked generated transcript, worker summary, factory report, and event log preserve the successive representations of this recording. **S5** retains the recording identifier and local offsets alongside shifted timestamps. Audit event 41 judges the leveling event

Table 16. Two separate audit samples against generated transcripts: 42 event-support judgments and 42 transcript-entry-overlap judgments. Strict support divides 36 fully supported events by all 42 sampled events. Entry overlap divides 34 overlapping entries by the 38 retained entries, including four uncovered operational entries and excluding four uncovered non-operational entries.

Sample outcome	Count	Interpretation
Event fully supported (C)	36	Strict support from generated text: 36/42, or 85.7%
Event partially supported (P)	6	Context is plausible; overlapping description is insufficient
Event contradicted (X)	0	No contradiction among the sampled events
Transcript entry overlaps (COV)	34	At least one event overlaps; action completeness not established
Uncovered, non-operational (EXC)	4	Excluded from the archive's reported overlap denominator
Uncovered, operational (MISS)	4	Reported entry-overlap rate: 34/38, or 89.5%

Table 17. Reproduced time analysis on 320 logs. Left: percentage of summed event duration. Right: medians across eligible workers after the original worker-level rounding. Trans./h means VA-to-non-VA transitions per summed event-hour. Mean VA run is the median of worker mean VA-run durations in minutes. A run sums consecutive same-category rows within a clip; transition counting can cross clips.

	Process-based category shares (%)							Medians across eligible workers		
Factory	VA	MH	TR	QI	CM	RW	DOC	VA (%)	Trans./h	Mean VA run (min)
001	93.1	4.1	1.8	0.0	0.2	0.0	0.9	94.5	9.2	4.6
002	96.3	0.1	2.3	0.2	0.0	0.0	1.1	97.3	3.9	8.8
003	82.1	0.4	0.0	0.0	6.3	10.8	0.4	86.4	3.6	9.0
004	89.9	3.0	3.5	2.0	0.9	0.0	0.7	91.3	10.6	3.6
005	80.6	5.4	9.7	4.3	0.0	0.0	0.0	85.9	5.6	6.2
006	74.2	12.1	7.8	0.0	5.7	0.0	0.3	87.0	14.4	3.1

fully supported by its generated transcript. This supports the label's textual basis, not the exact internal split or a video-based duration measurement.

*7.4. **RQ3**: Descriptive analytical capabilities*

7.4.1. Time composition and category transitions

We summarize the time assigned to each category and the pattern of VA runs for eligible workers. Table 17 separates pooled duration shares from medians across workers.

Table 17 reproduces the existing category shares using the two-log exclusion rule. For factory **006**, 74.2% pools the durations of all retained events, whereas 87.0% is the median of the eligible workers' individual VA shares. Combined shares are calculated before rounding, so they may differ slightly from the sum of the displayed percentages. For example, the combined MH/TR share for factory **006** is 19.8%, while its separately rounded components are 12.1% and 7.8%.

The non-VA category share ranges from 3.7% to 25.8% of retained summed event duration. This is neither total worker time nor recoverable capacity. Factory **003** assigns 10.8% to defect patching. Its factory report describes defect patching in the opening overview and vocabulary. The duration share quantifies this recorded behavior under an explicit category rule.

Figure 6 displays the category shares. The mean VA-run column is the median across workers of their *mean* VA-run durations. A value of 3.1 minutes in factory **006** is a central tendency; it says nothing about the longest VA episode. Factory **001** has 449 MH/TR rows, a mean row duration of 14.54 seconds, and 14.6 such rows per summed event-hour. Factory **006** has 192 MH/TR rows, a mean of 25.14 seconds, and 28.4 rows per summed event-hour.

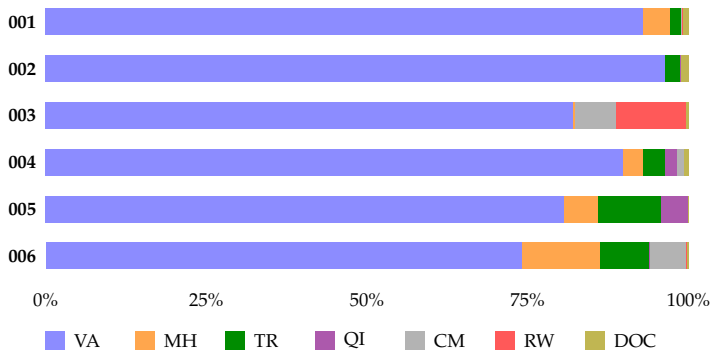


Figure 6. Category composition calculated from retained event rows and the released process map. Segments use unrounded duration totals; Table 17 reports rounded percentages. Factories have different task mixes and label schemes, so the plot does not rank efficiency.

These are row-based descriptions. Segmentation, repeated-action compression, and clip joins influence their values.

7.4.2. Process flow and discovery

Video-as-case traces describe observation windows. Every recording has a distinct activity sequence in factories **001**, **004**, **005**, and **006**. Factory **002** has 53 distinct sequences across 54 recordings, and factory **003** has 70 across 72. Self-loops account for 4.3% to 16.8% of within-video directly-follows pairs. These statistics are reproduced directly from the stored labels. A self-loop can reflect separately extracted repetitions or segmentation choices; it is not a count of manufactured items.

The released discovery script uses Inductive Miner [32] with noise threshold 0.2, video cases, and activity labels. Appendix C of the Additional Material records the repeated calculation and current dependency version. The recalculated fitness ranges from 0.9544 to 0.9910 and precision from 0.0603 to 0.2679. Together, these results show that the discovered models can replay most behavior in the inferred logs while also permitting substantial additional behavior. Fitness measures how the discovered model can replay the inferred traces; precision measures how much additional behavior it permits relative to that log. Both are properties of a model-log pair, not accuracy against actual factory behavior.

Many distinct traces and low precision can arise from flexible work, recording boundaries, abstraction choices, label variation, or extraction mistakes. The present evidence does not separate these causes. No process-level discovery result is reported because its computation and saved output are unavailable. Replacing activity labels with process labels would change the represented behavior and evaluation target. A higher precision after that change would not by itself be a controlled demonstration of better extraction or modeling.

7.4.3. Worker profiles and shared dominant processes

A shared vocabulary permits descriptive comparison within a factory. Table 18 groups workers by their largest process-duration total, then compares VA shares over *all* retained events for those workers. Eligibility requires at least 1,800 seconds of summed event duration. It does not require verified footage duration or equivalent tasks. There are 11, 10, 10, 10, 5, and 5 eligible workers in factories **001** to **006**.

The 29.6 percentage-point range in the surface-finishing group describes different observed profiles. Differences may reflect task mix, product requirements, recording selection, category assignment, or working conditions. Likewise, the small range for two lathe workers cannot rule out differences in behavior. Neither result supports individual performance rankings.

Table 18. Selected shared-dominant-process groups, using the original eligibility and rounding rules. Ranges summarize worker profiles; they are not estimates of individual efficiency.

Factory	Dominant process	Workers	VA share	Mean VA run
001	Manual mechanical assembly	6	89.7 to 97.8%	1.7 to 6.7 min
001	Metal stamping	3	90.6 to 98.5%	3.2 to 7.9 min
003	Surface finishing and polishing	5	67.3 to 96.9%	3.3 to 13.7 min
004	Garment ironing	2	52.8 to 77.1%	3.5 to 4.1 min
004	Overlock seaming	4	87.8 to 96.5%	1.5 to 10.3 min
005	Manual lathe machining	2	95.0 to 95.8%	6.9 to 9.4 min

7.5. Six descriptive factory cases

Each case below states its recorded scope, quantitative result, relationship to the report, and unresolved interpretation. Factory descriptions come from the generated reports; the logs quantify behavior described there. Different vocabularies, products, task mixes, and observation windows limit comparisons. The cases describe recorded work and motivate further study, but do not establish efficiency or the benefits of an intervention.

Factory **001**: hardware stamping and assembly.

The archive has 97 clips from 11 workers; the time analysis retains 95 clips. Manual mechanical assembly accounts for 55.7% of retained summed duration, metal stamping for 18.2%, and component bagging for 11.3%. The report describes material replenishment, floor storage, and operator transport. The 449 MH/TR rows and their short mean duration motivate examining how materials are supplied to workers. Product mix and compressed repetitions could also explain this pattern.

Factory **002**: garment finishing and packaging.

There are 54 clips from ten workers, all retained by the event-end rule. Fabric folding, garment bagging, and ironing contribute 18.8%, 17.1%, and 13.2% of summed duration. VA is 96.3%. The report describes transport, floor staging, and varied folding methods. The small separate handling and transport share may leave those actions counted within other processes. Two anomalous generated transcript endpoints and mixed formats weaken temporal interpretation. Product-specific finishing requirements and category boundaries need closer examination.

Factory **003**: surface preparation and assembly.

The archive covers 72 clips from 11 workers; all clips are retained. Surface finishing and polishing contributes 42.1%, wax pattern assembly 19.2%, and defect patching 10.8%. The report identifies patching and material transport. The log quantifies patching's share without establishing defect rates or avoidable rework. Its 101 transport events total 1,230 seconds, yet the process-based TR share is zero because their durations enter other categories. Material types and quality criteria remain uncertain in the report itself.

Factory **004**: garment sewing and finishing.

There are 51 clips from ten workers, all retained. Overlock seaming contributes 23.5%, garment folding 11.0%, and thread trimming 10.8%. The median worker VA-transition rate is 10.6 per summed event-hour. The report describes tool switching, material handling, and inspection. The logs add a category-transition measure and a 24.3 percentage-point VA-share range in the two-worker ironing group. Product specifications, additional finishing duties, and recording windows could explain this difference.

Factory **005**: machining and motor assembly.

The archive contains 22 clips from 11 workers, but only five workers meet the time threshold. All clips are retained, so the worker medians describe a smaller subset than the factory duration shares. Manual lathe machining contributes 32.1%, drill press machining 19.1%, and material transport and logistics 9.7%. The report describes staging, fixturing, inspection, and coupled workstations. The log assigns 4.3% to dimensional inspection. The two lathe profiles have similar VA shares, but their task equivalence is unknown. One generated transcript also has an unresolved endpoint anomaly.

Factory **006**: molding and foundry work.

There are 26 clips from six workers, all retained; five workers meet the time threshold. Sand mold creation, pattern cleaning and coating, and compression molding contribute 17.4%, 14.0%, and 12.2%. The report describes floor staging, material movement, cleaning, and mold-track operations. The logs assign 19.8% of summed duration to MH/TR and yield a median worker mean VA run of 3.1 minutes. These quantities support closer study of handling requirements. Necessary mold preparation, differing materials, layout, and segmentation choices could all contribute. Two generated transcript endpoints require resolution against source metadata.

7.6. From observations to operational hypotheses

Table 19 connects observations to explanations and the evidence needed to distinguish them. Reports identify phenomena; logs quantify them under a chosen representation. Differences in task mix, observation windows, and extraction choices remain competing explanations. Establishing causes or comparing efficiency requires matched operational evidence. Transport, inspection, cleaning, and documentation are not automatically waste, and their time is not automatically available for production. No intervention effect or cost saving is demonstrated by these descriptive results.

8. Measurement and Analytical Implications

8.1. Observability and the interpretation of zeros

A zero in an analytical table has several possible causes (Table 20). These causes occur at different stages and require different checks. Requiring a label for transport or waiting improves representability, but does not establish reliable detection or a measured absence. A model may still omit the action, assign another label, or absorb it into a longer episode.

In factory **003**, both *transport loaded tray to storage rack* and *transport bulk material via wheelbarrow* are permitted activity labels and occur in the logs. Together they account for 101 events and 1,230 seconds, yet the TR-category share is zero. Transport remains in the event log, but its duration is counted under other categories because the rule uses broader process labels and no process in this factory maps to TR. The zero describes the category map's output, not an absence of transport or a missing activity label.

To interpret an observed zero as evidence of absence within a window, an analyst would need a defined observation scope, a suitable representation, adequate detection, correct assignment, and an aggregation rule that preserves the behavior. None follows

Table 19. Evidence-linked hypotheses. These are candidates for investigation; intervention effects were not measured.

Observation	Plausible explanation	Competing explanation	Evidence needed
001: 449 short MH/TR rows; 5.9% of summed duration	Material presentation creates frequent replenishment	Segmentation splits one handling episode into several rows; task mix differs	Video checks, supply distances, batch quantities, station observations
006: 19.8% MH/TR; frequent category transitions	Layout or replenishment arrangements require repeated movement	Heavy or fragile materials require necessary support work	Routes, loads, safety requirements, matched task observations
003: 10.8% defect patching	Upstream defects require corrective work	Patching is a planned finishing requirement	Product specifications, defect criteria, item counts, upstream records
003 and **004**: VA shares differ within dominant-process groups	Conditions or work allocation differ	Product mix, selection, or extraction differences	Comparable products and observation windows; independent video review
005: two similar lathe VA shares	Common station requirements shape profiles	Similar aggregate shares hide different tasks or behavior	Task-level comparisons and production context

Table 20. Possible meanings of an apparent zero, ordered along the evidence path. Only the last mechanism is directly demonstrated by the factory **003** example; the others are possible mechanisms implied by the pipeline.

Mechanism	Where information is lost	Needed check
Behavior absent from the recorded window	Observation selection	Define the window and verify that it is suitable for the question
Behavior present but not narrated	Video interpretation or narration	Independent review of source footage
Behavior narrated but lost in summaries	**S2** or **S3** compression	Compare generated transcripts with worker and factory summaries
No suitable vocabulary label	Vocabulary induction	Inspect lists against the behavior of interest
Suitable label exists but is not assigned	Event extraction	Compare relevant generated transcript spans and extracted labels
Event exists but category aggregation hides it	Analytical mapping	Compare activity-level rows with process-derived categories

from label membership alone. Conversely, an observed event can establish what the representation contains while leaving the event's visual accuracy unmeasured.

8.2. Uncertainty propagation and limits of provenance

Several generated outputs can agree because they inherited the same earlier mistake. A mistaken object description in **S1** can shape the worker summary, the factory vocabulary, and the final event label along both branches of Figure 2. Provenance helps trace how an event was constructed, so readers can inspect these dependencies. Table 21 links the possible errors to their analytical consequences and the available evidence.

Quality-aware construction [9] and event-log imperfection patterns [23] establish the broader concern. The table separates observed examples from possible mechanisms; it does not estimate how often each error occurs.

These mechanisms occur within an observation scope that is itself incomplete: the selection protocol and source durations are unavailable. No stage-wise error rates are calibrated, and structural checks cannot quantify visual errors. Worker and factory differences may therefore reflect observation selection as well as representational choices.

Readable text is useful evidence, but it is weaker than structured event provenance. Existing records identify a source file and local interval. They do not store supporting generated transcript-span identifiers, alternative labels, calibrated confidence, or a flag that distinguishes an inferred internal boundary from a copied generated transcript endpoint. Finding the source of an event therefore requires reading and matching intervals. The foundry example can be traced manually, yet its CSV does not record why the split at 00:04:24 was chosen.

A future schema could add generated transcript-entry references, a boundary-origin field, and vocabulary and prompt versions. Data-quality annotations could retain known issues for downstream analysis [24]. Explicit alternative labels or bounded times would instead extend the event representation toward uncertain data [25,33]. Such alternatives need evidence: the five-second matching tolerance does not imply that an event boundary lies within five seconds of its true value. Calibrated confidence values are unavailable for the existing events. Provenance can explain how a conclusion was reached; it does not certify that every preceding observation was correct.

8.3. Missing time does not imply a one-sided bound

The observed share outside VA is sometimes described as a lower bound because waiting and other supporting work can be omitted. That interpretation also assumes that omitted time is sufficiently non-VA. Missing VA time can instead raise the observed non-VA

Table 21. Transformation, possible imperfection, analytical consequence, and existing evidence. A prompt or a permitted operation establishes a possible mechanism, not its error frequency.

Transformation	Possible imperfection	Downstream consequence	Existing evidence
S1: narration	Omitted actions or compressed repetitions; inaccurate times	Row frequency differs from cycle frequency; duration can be distorted	Repetition rule in prompt; seven endpoint anomalies in Table 11; visual omissions unmeasured
S2 and **S3**: summaries and classes	Rare details lost; concepts merged or separated	Vocabulary may omit a needed distinction	Compression requested by prompts; lists available for inspection; information loss not quantified
S4: label assignment	Episode forced into the closest permitted label	Membership passes despite weak textual support	Six partially supported audit events, including the bagging example; mechanism not isolated by the audit
S4: interval assignment	Internal boundary added without a timed transition in the text	Duration attributed to each action depends on the split	Foundry split at 00:04:24; 35 adjacent overlaps in the full archive
S5: consolidation	Filename joins treated as continuous behavior	Added transitions and synthetic elapsed time	Merge code and preserved local offsets; real inter-clip gaps unknown
Analysis: category mapping	Recorded activity hidden by its process category	Apparent zero for a behavior present in the log	Factory **003** has 101 transport events but no process mapped to TR

share. We consider the two extreme possibilities: all missing time is VA, or all missing time is non-VA.

Proposition.

Consider an idealized observation with $C > 0$ seconds of correctly classified, non-overlapping observed time, of which N seconds are non-VA. Let $U \geq 0$ seconds be unobserved. If observed and unobserved time partition a known target window, then its full-time non-VA share satisfies

$$\frac{N}{C+U} \leq p_{\text{nonVA}} \leq \frac{N+U}{C+U}. \tag{4}$$

Proof. Let x be the non-VA duration in the unobserved part. Since $0 \leq x \leq U$, the full share is $(N+x)/(C+U)$ and has the stated extrema. Both are attainable: all missing time may be VA or all may be non-VA.

The lower endpoint assigns all missing time to VA; the upper endpoint assigns all of it to non-VA. For $U > 0$, the observed ratio N/C is a lower bound only if $x/U \geq N/C$: the missing time must contain at least the observed proportion of non-VA time. Thus, merely knowing that some waiting is missing is insufficient. The interval width in Equation 4 is $U/(C+U)$, the missing fraction of the target window. It shows how unknown coverage limits a share even with perfect observed labels.

As an illustration, suppose 80 seconds are observed, 20 of them are non-VA, and 20 seconds are missing. The observed non-VA share is 25%. Over the full 100-second window, it could be as low as 20% if all missing time is VA, or as high as 40% if all missing time is non-VA.

This is a conceptual result, not an empirical confidence interval for this dataset. Source durations and therefore U are unavailable; labels are inferred; and summed event durations include overlaps. Substituting the coverage diagnostic or the 97.1 retained summed event-hours into this proposition would violate its assumptions. The result identifies the additional evidence needed before making a bound claim.

8.4. Why aggregation changes the question

A duration-weighted factory share estimates the composition of the retained event rows. A median worker share gives equal weight to each eligible worker. They answer different questions, as the factory **006** example beside Table 17 shows. Different observa-

tion amounts and work profiles can produce this difference without either statistic being erroneous.

Likewise, process-based and activity-based categories need not agree. Counting rows emphasizes the extraction granularity; counting runs suppresses adjacent labels; measuring interval unions removes overlap but does not resolve classification conflicts. None is a universal correction. A defensible analysis states its unit, denominator, case rule, handling of gaps, and interpretation before drawing operational conclusions. The released measurements serve as a reproducible example of those choices and their limits.

9. Discussion and Conclusions

9.1. Main findings

Egocentric recordings can be organized into a shared representation of manual work through generated descriptions, shared factory labels, and interval-event extraction. For **RQ1**, the released logs satisfy the reported field, ordering, and vocabulary checks. For **RQ2**, 36 of 42 sampled events were fully supported by their generated descriptions, while accuracy against the videos remains unmeasured. These results distinguish structural consistency and textual support from correct observation.

For **RQ3**, the logs support descriptions of work composition, local sequences, and worker profiles. They quantify behaviors described in the reports and identify questions for further investigation. For **RQ4**, the examples show how representation choices affect these measurements. In factory **003**, the process map counts recorded transport under other categories. In the foundry example, an inferred boundary divides a described episode between activities. Synthetic clip joins can add transitions, and missing time can move a category share in either direction. Interpreting a result therefore requires its category rule, event boundaries, case definition, and denominator.

9.2. Limits and responsible use

The vocabulary is induced from the same factory corpus used for assessment, so the results do not establish transfer to unseen factories. The single-author audit uses generated text and cannot reveal visual errors shared by the generated transcripts and logs.

Missing source durations prevent us from measuring footage coverage or the amount of missing time. Missing selection records prevent us from establishing how representative the processed recordings are. Missing historical execution details prevent exact regeneration of the model outputs. The released artifacts still permit reproduction of the descriptive calculations.

Different task mixes, observation windows, and category assignments can explain differences between worker profiles. Product identifiers and production outcomes are absent, and no intervention was evaluated. The results consequently support hypotheses about process design, not causal diagnoses, efficiency rankings, or estimates of recoverable capacity.

Worker identifiers are pseudonymous and may still enable identification when combined with local information [34]. Use should remain tied to an agreed analytical purpose, with suitable aggregation and review by the people whose work is represented. These inferred profiles do not support employee appraisal, pay, or disciplinary decisions. The source card does not document the original consent procedure; this secondary study collected no new consent.

9.3. Next steps

The linked generated transcripts, summaries, vocabularies, and logs provide a basis for testing alternative event representations. Priorities are independent video assessment with multiple evaluators, preservation of source durations, and explicit links from events to supporting generated transcript spans. Product references and real calendar times would be needed to extend the resource to synchronized production-flow analysis.

Additional Material

The accompanying Additional Material contains the evidence and assessment scope (Appendix A), reproducibility information and configuration (Appendix B), activity-level discovery results (Appendix C), the complete operational-category map (Appendix D), and all archived audit judgments (Appendix E). The four full prompt templates appear in Listings 2 to 5 in Section 4.

Declarations

Author Contributions: Alessandro Berti carried out the experiments. Wil M. P. van der Aalst reviewed the manuscript and suggested improvements.

Funding: Funded by the European Union. This work received funding from the European High Performance Computing Joint Undertaking (JU), the German Federal Ministry of Research, Technology and Space (BMFTR), the Ministry of Culture and Science of North Rhine-Westphalia (MKW NRW), and the Hessian Ministry of Science and Research, Arts and Culture (HMWK), under grant agreement No. 101250682.

Institutional Review Board Statement: This study is a secondary analysis of the existing Egocentric-10K dataset. It involved no recruitment of participants, intervention, or new video collection. No ethics approval or exemption record for this secondary analysis is reported.

Informed Consent Statement: No participants were recruited or contacted for this secondary analysis. The source videos were obtained from the existing Egocentric-10K dataset. The authors did not collect new consent, and the source dataset card does not report the original consent procedure.

Data Availability Statement: The pipeline, prompts, generated transcripts, factory vocabularies, derived event logs, and assessment code are available at https://github.com/fit-alessandro-berti/annotated-egocentric-10k-dataset. The source videos are available from Build AI's Egocentric-10K dataset at https://huggingface.co/datasets/builddotai/Egocentric-10K, under its access terms and Apache License 2.0. The repository contains derived artifacts and does not redistribute the source videos.

Acknowledgments: The study used Gemini 3.1 Pro for video descriptions, worker summaries, and factory reports, and GPT-5.4 for event extraction, as described in the methods. OpenAI Codex (GPT-6) assisted with manuscript preparation, evidence checks, descriptive recalculation scripts, language editing, LaTeX formatting, and preparation of the cover letter. The authors are responsible for the content of this work.

Conflicts of Interest: Wil M. P. van der Aalst is affiliated with Celonis, a process mining software company. No other conflicts of interest are reported.

References

1. van der Aalst, W.M.P. *Process Mining: Data Science in Action*, 2 ed.; Springer, 2016. https://doi.org/10.1007/978-3-662-49851-4.
2. Kratsch, W.; König, F.; Röglinger, M. Shedding light on blind spots - Developing a reference architecture to leverage video data for process mining. *Decis. Support Syst.* **2022**, *158*, 113794. https://doi.org/10.1016/j.dss.2022.113794.
3. Build AI. Egocentric-10K: Dataset Card. https://huggingface.co/datasets/builddotai/Egocentric-10K, 2025. Accessed 19 October 2026; current revision 3e5f87c.

4. Petersen, N.; Pagani, A.; Stricker, D. Real-Time Modeling and Tracking Manual Workflows from First-Person Vision. In Proceedings of the 2013 IEEE International Symposium on Mixed and Augmented Reality (ISMAR). IEEE, 2013, pp. 117 to 124. https://doi.org/10.1109/ISMAR.2013.6671771.
5. Pham, T.; Xiao, Y. Unsupervised Workflow Extraction from First-person Video of Mechanical Assembly. In Proceedings of the 19th International Workshop on Mobile Computing Systems and Applications. ACM, 2018, pp. 31 to 36. https://doi.org/10.1145/3177102.3177112.
6. Gavric, A.; Bork, D.; Proper, H. Multimodal Process Mining. In Proceedings of the 2024 IEEE 26th Conference on Business Informatics (CBI). IEEE, 2024, pp. 99 to 108. https://doi.org/10.1109/CBI62504.2024.00021.
7. König, F.; Egger, A.; Kratsch, W.; Röglinger, M.; Wördehoff, N. Unstructured Data in Process Mining: A Systematic Literature Review. *ACM Trans. Manag. Inf. Syst.* **2025**, *16*, 25:1 to 25:34. https://doi.org/10.1145/3727148.
8. Diba, K.; Batoulis, K.; Weidlich, M.; Weske, M. Extraction, correlation, and abstraction of event data for process mining. *WIREs Data Mining Knowl. Discov.* **2020**, *10*, e1346. https://doi.org/10.1002/widm.1346.
9. Andrews, R.; van Dun, C.G.J.; Wynn, M.T.; Kratsch, W.; Röglinger, M.; ter Hofstede, A.H.M. Quality-Informed Semi-Automated Event Log Generation for Process Mining. *Decis. Support Syst.* **2020**, *132*, 113265. https://doi.org/10.1016/j.dss.2020.113265.
10. Kecht, C.; Egger, A.; Kratsch, W.; Röglinger, M. Event Log Construction from Customer Service Conversations Using Natural Language Inference. In Proceedings of the 2021 3rd International Conference on Process Mining. IEEE, 2021, pp. 144 to 151. https://doi.org/10.1109/ICPM53251.2021.9576869.
11. Pegoraro, M.; Seng, J.; Heller, D.; van der Aalst, W.M.P.; Kersting, K. All Eyes on the Workflow: Automated and Efficient Event Discovery from Video Streams. *CoRR* **2026**, *abs/2604.22476*. https://doi.org/10.48550/arXiv.2604.22476.
12. Seeth, M.; Tavares, G.M.; Schuster, D. Automated Event Log Generation from Unstructured Text Using Finetuned LLMs, 2026, [2609.01320]. Preprint.
13. Knoch, S.; Ponpathirkoottam, S.; Schwartz, T. Video-to-Model: Unsupervised Trace Extraction from Videos for Process Discovery and Conformance Checking in Manual Assembly. In Proceedings of the BPM. Springer, 2020, Vol. 12168, *Lecture Notes in Computer Science*, pp. 291 to 308. https://doi.org/10.1007/978-3-030-58666-9_17.
14. Lepsien, A.; Koschmider, A.; Kratsch, W. Analytics Pipeline for Process Mining on Video Data. In Proceedings of the BPM (Forum). Springer, 2023, Vol. 490, *Lecture Notes in Business Information Processing*, pp. 196 to 213. https://doi.org/10.1007/978-3-031-41623-1_12.
15. Brzychczy, E.; Aleknonytė-Resch, M.; Janssen, D.; Koschmider, A. Process mining on sensor data: a review of related works. *Knowl. Inf. Syst.* **2025**, *67*, 4915 to 4948. https://doi.org/10.1007/s10115-024-02297-y.
16. Krishna, R.; Hata, K.; Ren, F.; Fei-Fei, L.; Niebles, J.C. Dense-Captioning Events in Videos. In Proceedings of the IEEE International Conference on Computer Vision (ICCV). IEEE, 2017, pp. 706 to 715. https://doi.org/10.1109/ICCV.2017.83.
17. Shirali, M.; Fani Sani, M.; Ahmadi, Z.; Serral, E. LLM-Based Event Abstraction and Integration for IoT-Sourced Logs. In Proceedings of the Business Process Management Workshops. Springer, 2025, Vol. 534, *Lecture Notes in Business Information Processing*, pp. 138 to 149. https://doi.org/10.1007/978-3-031-78666-2_11.
18. Berti, A.; Kourani, H.; van der Aalst, W.M.P. PM-LLM-Benchmark: Evaluating Large Language Models on Process Mining Tasks. In Proceedings of the ICPM Workshops. Springer, 2025, Vol. 533, *Lecture Notes in Business Information Processing*, pp. 610 to 623. https://doi.org/10.1007/978-3-031-82225-4_45.
19. Dani, V.S.; Dees, M.; Leopold, H.; Busch, K.; Beerepoot, I.; van der Werf, J.M.E.M.; Reijers, H.A. Event Log Extraction for Process Mining Using Large Language Models. In Proceedings of the CoopIS. Springer, 2025, Vol. 15506, *Lecture Notes in Computer Science*, pp. 56 to 72. https://doi.org/10.1007/978-3-031-81375-7_4.
20. van Zelst, S.J.; Mannhardt, F.; de Leoni, M.; Koschmider, A. Event Abstraction in Process Mining: Literature Review and Taxonomy. *Granular Computing* **2021**, *6*, 719 to 736. https://doi.org/10.1007/s41066-020-00226-2.
21. Kapugama Geeganage, D.T.; Wynn, M.T.; ter Hofstede, A.H.M. Text2EL+: Expert Guided Event Log Enrichment Using Unstructured Text. *ACM J. Data Inf. Qual.* **2024**, *16*, 8:1 to 8:28. https://doi.org/10.1145/3640018.
22. Sturm, F.; Liessmann, A.; Matzner, M. Multimodal Event Log Construction for Process Mining: Instantiating a Reference Architecture. In Proceedings of the Design for Better Futures: Beyond the Science of the Artificial. Completed Research. Springer, 2026, Vol. 16605, *Lecture Notes in Computer Science*, pp. 146 to 164. https://doi.org/10.1007/978-3-032-28316-0_9.
23. Suriadi, S.; Andrews, R.; ter Hofstede, A.H.M.; Wynn, M.T. Event Log Imperfection Patterns for Process Mining: Towards a Systematic Approach to Cleaning Event Logs. *Information Systems* **2017**, *64*, 132 to 150. https://doi.org/10.1016/j.is.2016.07.011.
24. Goel, K.; Leemans, S.J.J.; Martin, N.; Wynn, M.T. Quality-Informed Process Mining: A Case for Standardised Data Quality Annotations. *ACM Trans. Knowl. Discov. Data* **2022**, *16*, 97:1 to 97:47. https://doi.org/10.1145/3511707.
25. Pegoraro, M.; Uysal, M.S.; van der Aalst, W.M.P. Conformance Checking over Uncertain Event Data. *Information Systems* **2021**, *102*, 101810. https://doi.org/10.1016/j.is.2021.101810.

26. Esser, S.; Fahland, D. Multi-Dimensional Event Data in Graph Databases. *J. Data Semant.* **2021**, *10*, 109 to 141. https://doi.org/10.1007/s13740-021-00122-1.
27. van der Aalst, W.M.P. Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data. In Proceedings of the SEFM. Springer, 2019, Vol. 11724, *Lecture Notes in Computer Science*, pp. 3 to 25. https://doi.org/10.1007/978-3-030-30446-1_1.
28. Corallo, A.; Lazoi, M.; Striani, F. Process Mining and Industrial Applications: A Systematic Literature Review. *Knowledge and Process Management* **2020**, *27*, 225 to 233. https://doi.org/10.1002/kpm.1630.
29. Sener, F.; Chatterjee, D.; Shelepov, D.; He, K.; Singhania, D.; Wang, R.; Yao, A. Assembly101: A Large-Scale Multi-View Video Dataset for Understanding Procedural Activities. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 21096 to 21106.
30. Gebru, T.; Morgenstern, J.; Vecchione, B.; Vaughan, J.W.; Wallach, H.; Daumé III, H.; Crawford, K. Datasheets for Datasets. *Communications of the ACM* **2021**, *64*, 86 to 92. https://doi.org/10.1145/3458723.
31. Berti, A.; van Zelst, S.J.; Schuster, D. PM4Py: A process mining library for Python. *Softw. Impacts* **2023**, *17*, 100556. https://doi.org/10.1016/j.simpa.2023.100556.
32. Leemans, S.J.J.; Fahland, D.; van der Aalst, W.M.P. Discovering Block-Structured Process Models from Event Logs - A Constructive Approach. In Proceedings of the Petri Nets. Springer, 2013, Vol. 7927, *Lecture Notes in Computer Science*, pp. 311 to 329. https://doi.org/10.1007/978-3-642-38697-8_17.
33. Pegoraro, M.; van der Aalst, W.M.P. Mining Uncertain Event Data in Process Mining. In Proceedings of the 2019 International Conference on Process Mining. IEEE, 2019, pp. 89 to 96. https://doi.org/10.1109/ICPM.2019.00023.
34. Mannhardt, F.; Petersen, S.A.; Oliveira, M.F. Privacy Challenges for Process Mining in Human-Centered Industrial Environments. In Proceedings of the Intelligent Environments. IEEE, 2018, pp. 64 to 71. https://doi.org/10.1109/IE.2018.00017.